

Scripting

Create, attach and implement gameplay scripts in native C++ or Lua. The same gameplay behavior runs in the editor's Play mode and in packaged games, with access to animation, timers, character movement states, particles, audio, UI and world systems.

[engine::Script](#)[Lifecycle](#)[Register in game module](#)[Compile & Restart](#)[Attach + fields](#)[The API](#)[Crouch + swim states](#)[Worked examples](#)

How scripting works

A native script is a C++ class derived from `engine::Script` ; a Lua script is a `.lua` file with the same lifecycle callbacks. Both attach through the Script component, support tunable fields and run in editor Play and packaged games. Native scripts compile into the shared `game` library; Lua scripts reload when Play starts and require no compilation.

The lifecycle

Method	When it runs
<code>OnCreate()</code>	Once, when the script starts (entering Play / scene load)
<code>OnUpdate(float dt)</code>	Once per rendered frame (variable <code>dt</code>) - most logic goes here
<code>OnFixedUpdate(float dt)</code>	Once per physics step (fixed <code>dt</code>) - physics-coupled logic
<code>OnDestroy()</code>	Once, before the entity/script is destroyed

TWO WAYS TO AUTHOR You can write native scripts **in code** (a header under `game/` , registered in the game module), or create either C++ or Lua from the editor's built-in **Script templates**. Use **Compile & Restart** for C++; save Lua and restart Play to reload it.

1 · Write the script class

Create a header under `game/include/game/scripts/`. The minimal example (this exact one ships as `Spinner`) rotates its object every frame:

```
#pragma once
#include <engine/gameplay/Script.h>
#include <engine/ecs/Components.h>
#include <glm/gtc/quaternion.hpp>

class Spinner : public engine::Script {
public:
    void OnUpdate(float dt) override {
        if (engine::ecs::Transform* t = Transform()) {
            const glm::quat spin = glm::angleAxis(dt * 1.5f, glm::vec3(0,1,0));
            t->rotation = glm::normalize(spin * t->rotation);
        }
    }
};
```

- `Transform()` returns this object's transform (or nullptr).
- `Self()` is this entity; `Registry()` is the world. Both are available anywhere in the class.
- Override only the lifecycle methods you need. If the script has a `.cpp`, list it in `game/CMakeLists.txt`; header-only scripts need nothing extra.

2 · Register it in the game module

Scripts are looked up by name at runtime. Register yours in `game/src/GameModule.cpp` :

```
// 1. include your header near the top:
#include "game/scripts/Spinner.h"

void RegisterGameModule() {
    engine::ScriptRegistry& scripts = engine::ScriptRegistry::Instance();

    // 2. register it. The name MUST match what a scene stores (the class name
    //     you'll type in the editor's Script field).
    scripts.Register("Spinner", [] { return std::make_unique<Spinner>(); });
}
```

THE NAME MUST MATCH The string you pass to `Register("...")` is the identifier the editor stores on an object and looks up at runtime. It must exactly match the **Class Name** you set in the Inspector (Step 4). A mismatch means the object silently has no behavior.

3 • Compile

Because scripts are C++, changes take effect after a rebuild of the `game` library. From the editor you don't have to leave the app:

Editor > **Compile & Restart** (script tools)

This runs the bundled `3DGScripCompiler`, rebuilds the game module, and relaunches - the build log and status are written under `build/script_compile.log` / `.status` and surfaced in the editor. Alternatively rebuild the whole project from your normal CMake build. Either way, the compiled script now runs in **Play mode and the packaged player** identically.

ITERATION LOOP Edit the script → Compile & Restart → Play. Keep scripts small so each compile is quick.

LUA WORKFLOW Choose **Lua** in the Script component, create the script, save it under `Content/Scripts`, then restart Play. Lua scripts do not use `GameModule.cpp` and do not require compilation.

4 • Attach it to an object

1. Select an object, open the **Inspector**, and **Add Component** ► **Script**.
2. Set **Class Name** to the registered name (e.g. `Spinner`).
3. Enable the script. Press P to Play - the behavior runs.

The object stores the class name (and any fields) in the scene, so it's restored on load and carried into the packaged build.

5 • Expose tunable fields

Hard-coding values (speeds, ranges, targets) is limiting. Declare **fields** on the object's Script component in the Inspector - each has a name, a type (Float / Int / Bool / String) and a value - and read them in the script:

```
void OnUpdate(float dt) override {
    float speed = GetFieldFloat("spinSpeed", 1.5f); // reads the "spinSpeed" field
    bool active = GetFieldBool("active", true);
    if (!active) return;
    if (auto* t = Transform())
        t->rotation = glm::normalize(glm::angleAxis(dt*speed, glm::vec3(0,1,0)) * t->rotation);
}
```

Reader	Field type
<code>GetFieldFloat(name, fallback)</code>	Float
<code>GetFieldInt(name, fallback)</code>	Int
<code>GetFieldBool(name, fallback)</code>	Bool
<code>GetFieldString(name, fallback)</code>	String

Now the same script drives many objects with different values, tuned per-object in the editor - no recompile to change a number.

6 · The script API

Everything below is available inside your script (protected members of `engine::Script`).

This object & the world

<code>Self()</code> , <code>Registry()</code> , <code>Transform()</code>	this entity, the world, this transform
<code>FindObject(name)</code> , <code>FindTransform(name)</code>	look another object up by name
<code>TryGet<T>()</code> , <code>Has<T>()</code> , <code>Add<T>()</code> , <code>Remove<T>()</code>	access/add/remove components on self (overloads take an entity)
<code>SpawnEmpty(name,pos)</code> , <code>SpawnFromObject(prototypeName,pos)</code>	create objects at runtime
<code>DestroySelf()</code> , <code>Destroy(entity)</code>	remove objects

Input

<code>IsKeyDown(key)</code> , <code>WasKeyPressed(key)</code>	held / just-pressed this frame
<code>IsMouseButtonDown(b)</code> , <code>WasMouseButtonPressed(b)</code> , <code>MouseDeltaX/Y()</code>	mouse

Triggers, collision & timers

<code>WasTriggerEntered(e)</code> , <code>WasTriggerExited(e)</code> , <code>IsTriggerTouching(e)</code>	respond to trigger volumes
<code>SetTimer(seconds, cb, repeat)</code> , <code>Delay(seconds, cb)</code> , <code>ClearTimer(id)</code>	schedule callbacks
<code>BindTimerFunction(name, cb)</code> , <code>SetTimerByFunctionName(name, seconds, repeat)</code>	schedule a registered event/function by name
<code>ClearTimerByFunctionName(name)</code> , <code>IsTimerActive(id/name)</code>	manage named timers
<code>SetGlobalTimeDilation(scale)</code> , <code>HitStop(seconds, scale)</code>	slow motion and real-time impact freezes

Animation

<code>SetAnimationParameter(name,v)</code> , <code>SetAnimationBool</code> , <code>SetAnimationTrigger</code>	drive the animation graph (e.g. push <code>Speed</code>)
<code>GetAnimationParameter(name,fallback)</code> , <code>GetAnimationBool(name,fallback)</code>	read graph values and controller-driven movement flags from C++
<code>PlayAnimationAction(clip)</code> , <code>PlayMaskedAnimationAction(clip,rootBone)</code> , <code>PlayAnimationProfile(name)</code>	one-shot / layered actions (attacks, hits)
<code>WasAnimationEvent(name)</code> , <code>IsAnimationActionPlaying()</code> , <code>IsAnimationMovementLocked()</code>	react to notifies / action state

Audio, particles, camera

<code>PlayAudio()</code> , <code>PlayAudioCue(path)</code> , <code>StopAudio()</code> , <code>SetAudioVolume(v)</code> ...	this object's audio source or a one-shot cue
<code>PlayParticles()</code> , <code>BurstParticles(n)</code> , <code>StopParticles()</code> ...	this object's particle system
<code>ShakeCamera(intensity,duration)</code> , <code>PlayCameraSequence(name)</code>	camera feedback / cinematics

Persistence & flow

<code>SaveValue(key,value)</code> , <code>LoadValue(key,fallback)</code>	simple key/value save data
<code>SaveCheckpoint(name,pos)</code> , <code>LoadCheckpoint(name,&pos)</code>	checkpoints
<code>RequestSceneLoad(runtimeScenePath)</code>	load another scene (applied safely after the update)

7 · Character crouching and swimming

The Player Controller owns crouching and swimming. Configure these values on the character in the **Character Editor** ▶ **Movement + Camera**, or on a player object in **Inspector** ▶ **Player Controller**.

Setting	Purpose
Crouch Speed	Horizontal speed while the crouched capsule is active.
Crouched Height	Total crouched collision-capsule height, clamped between twice the radius and standing height.
Swim Speed	Forward and lateral swimming speed inside a Water object.
Vertical Swim Speed	Rise and descend speed.

Default controls

- Hold **Ctrl** or **C** to crouch. Releasing it stands only when head room is clear.
- Inside water, use **WASD** to swim, **Space** to rise, and **Ctrl** / **C** to descend.
- Sprint and ground jump are disabled while crouching or swimming.

Animation graph parameters

The controller updates these parameters automatically every fixed movement step. Do not duplicate this movement logic or resize its trigger proxy in a player script. Swimming activates only over a visible Water object.

Parameter	Type	Use
Speed	Float	Walk, run, crouch or swim movement speed.
VerticalSpeed	Float	Jump/fall velocity or vertical swimming velocity.
IsGrounded , IsFalling	Bool	Ground locomotion and airborne transitions.
IsCrouching	Bool	Enter or leave crouch locomotion.
IsSwimming	Bool	Switch to the swimming state machine or blend space.

Reacting from a native C++ script

```
void OnFixedUpdate(float) override {
    const bool crouching = GetAnimationBool("IsCrouching", false);
    const bool swimming = GetAnimationBool("IsSwimming", false);

    if (swimming && WasKeyPressed(GLFW_KEY_E)) {
        PlayAudioCue("Content/Audio/swim_interact.wav", false);
    }
    if (crouching) {
        // Quiet footsteps, stealth checks, or crouch-only interactions.
    }
}
```

8 · Worked examples

Pickup - collect on touch

```
class Pickup : public engine::Script {
public:
    void OnUpdate(float) override {
        // fires when the player (a trigger) enters this object
        if (WasTriggerEntered(Self())) {
            PlayAudioCue(GetFieldString("sound", "Content/Audio/coin.wav"), false);
            // ...award score / set a flag your goal checks...
            DestroySelf();
        }
    }
};
```

Door - open when triggered

```
class Door : public engine::Script {
    bool m_open = false;
public:
    void OnUpdate(float dt) override {
        if (!m_open && WasTriggerEntered(Self())) m_open = true;
        if (m_open) {
            if (auto* t = Transform()) {
                float target = GetFieldFloat("openHeight", 3.0f);
                t->position.y = glm::mix(t->position.y, target, dt * 4.0f);
            }
        }
    }
};
```

Key-press action

```
void OnUpdate(float) override {
    if (WasKeyPressed(GLFW_KEY_E)) { // interact
        if (auto* target = FindObject("Lever")) { /* ... */ }
    }
}
```

Remember to register each class in `GameModule.cpp` and set the matching Class Name on the object.

9 • The editor template path

For a faster loop, the editor's Script component can generate a starting script from a **template**, let you edit the source in-app, and **Compile & Restart** - no manual file editing. Built-in templates:

Template	Starting point for...
Empty	a blank script (just the lifecycle stubs)
Player Movement	reading input and moving the object
Door Opener	trigger-driven motion
Pickup	collect-on-touch + destroy
Damage Zone	apply damage on contact

Set **Language** to **Native C++** or **Lua** before creating. Both languages appear in the saved-script dropdowns. C++ uses **Compile & Restart**; Lua reloads the next time Play starts.

Generated C++ scripts are registered through the game module. Generated Lua scripts are saved under **Content/Scripts** and load directly when Play starts. Use templates to prototype, then hand-edit as the logic grows.

Gotchas & cheat-sheet

Pitfall	Fix
Object has no behavior in Play	Class Name must exactly match the <code>Register("...")</code> name, and the script must be enabled.
Changes don't take effect	For C++, use Compile & Restart . For Lua, save and restart Play.
Crouch/swim animation never changes	Add <code>IsCrouching</code> and <code>IsSwimming</code> Bool parameters and transitions to the animation graph.
Physics feels frame-rate dependent	Put physics-coupled logic in <code>OnFixedUpdate(dt)</code> , not <code>OnUpdate</code> .
Crash when loading a scene from a callback	Use <code>RequestSceneLoad(...)</code> - it is deferred to a safe point.
Field change requires recompile	It does not - expose it as a field and read via <code>GetFieldFloat/Int/Bool/String</code> .

Add-a-script checklist

1. Header in `game/include/game/scripts/` , subclass `engine::Script` .
2. `#include` + `scripts.Register("Name", ...)` in `GameModule.cpp` .
3. **Compile & Restart**.
4. Add a **Script** component to an object; set **Class Name**; add any **fields**.
5. P to Play.

Scripts run in the shared `game` module linked by both the editor and the standalone player, so what you test in Play mode is exactly what ships.